SCÉNÁŘ PŘEDNESU · KIV/PTE POKROČILÉ TESTOVÁNÍ · ZČU FAV

Coordinated omission

a pasti průměrných časů

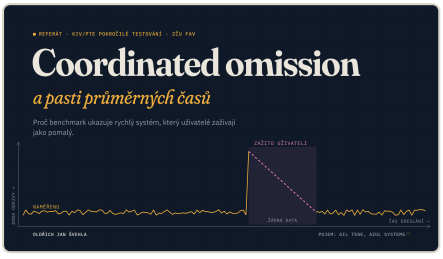
OLDŘICH JAN ŠVEHLA

15 SLIDŮ · PLÁN 10:00

ZVÝRAZNĚNÍ
Klíčová věta, kterou je dobré říct skoro doslova.

KLIK
▶ KLIK posune slide o krok (šipka → nebo mezerník).

ČAS
Kumulativní plán; v prezentaci ukazuje režim řečníka (klávesa P) i časovač.



01 0:00–0:55 · ~55 s

Úvod

Dobrý den. U zátěžových testů nás nejvíc zajímá doba odezvy. Dnes ukážu, **proč to číslo umí lhát, i když ho nikdo nefalšuje**. Příčiny jsou dvě: **coordinated omission**, česky koordinované vynechání, pojem od Gila Tene, a **průměry**. Graf na pozadí to ukazuje zjednodušeně. Vodorovně je čas, svisle doba odezvy. Žlutě je, co naměřil testovací nástroj: **jeden pomalý vzorek a pak nic**. Růžově je, co zažili uživatelé: kdo přišel na začátku výpadku, čekal nejdéle.

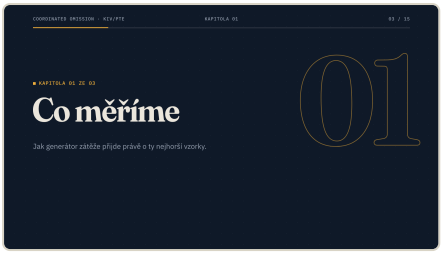


02 0:55–1:30 · ~35 s

Teze

Coordinated omission je **chyba v tom, co naměříte**: nástroj se zasekne spolu se systémem a nejhorší chvíle v datech chybí. To jsou prázdné kroužky na obrázku vlevo. Průměr je **chyba v tom, jak to shrnete**: schová ocas, tedy nejpomalejší odpovědi na obrázku vpravo.

▶ KLIK Zkrátka: **první kazí data, druhá závěry**.



03 1:30–1:35 · ~5 s

Předěl: Co měříme

Začnu měřením.



04 1:35–2:25 · ~50 s

Uzavřená smyčka

Většina nástrojů na zátěžové testy funguje **v uzavřené smyčce**, třeba běžná Thread Group v JMeteru. Každé vlákno pošle požadavek, počká na odpověď, zapíše čas a teprve pak pošle další.

▶ KLIK Teď se server zasekne. Vlákno čeká s ním a nakonec zapíše jeden pomalý vzorek.

▶ KLIK Jenže tyhle požadavky měly mezitím odejít. **Co nikdo neposlal, to nikdo nezměřil.** Z výpadku tak zbude **jeden pomalý vzorek mezi stovkami rychlých**.



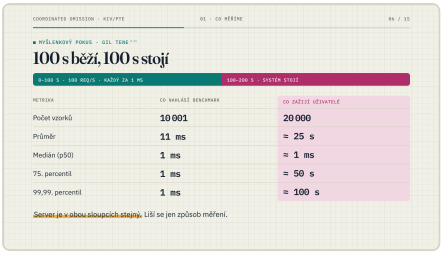
05 2:25–3:05 · ~40 s

Definice

Tene říká, že se systém a měřič nechtěně sladí: **když zpomalí systém, zpomalí i měření**.

▶ KLIK Je to jako anketa, která se ptá jen lidí odcházejících od přepážky. Když je přepážka zavřená, nikdo si nestěžuje.

▶ KLIK Test pak vidí hlavně dobu obsluhy. **Uživatel ale zažívá dobu odezvy: čekání ve frontě plus obsluhu.**



06 3:05–4:20 · ~75 s

Myšlenkový pokus: 100 s běží, 100 s stojí

Systém sto sekund odpovídá za milisekundu, pak sto sekund stojí. Nástroj nasbírá deset tisíc rychlých vzorků a jeden stosekundový. Průměr vyjde necelých jedenáct milisekund. A protože **99. percentil je hodnota, pod kterou leží 99 procent měření**, jeden pomalý vzorek z deseti tisíc s ním nepohne: **i 99,99. percentil je milisekunda**.

▶ KLIK Kdyby lidé chodili dál, přišlo by během výpadku dalších deset tisíc požadavků. Kdo přišel na začátku, čeká sto sekund, kdo na konci, skoro nic, **takže v průměru padesát**. Polovina požadavků je rychlá a polovina čeká padesát, **celkem tedy asi pětadvacet sekund**.

▶ KLIK **Server je v obou sloupcích stejný. Liší se jen měření.**



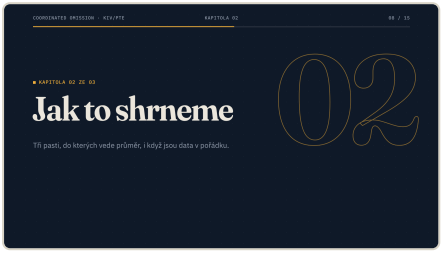
07 4:20–5:05 · ~45 s

wrk2: 200×

Tene napsal i wrk2, nástroj na zátěžové testy, který vypíše čísla dvakrát: jednou měřená od skutečného odeslání, jednou od plánovaného. Tomu druhému se říká korekce. V jeho příkladu server na 1,4 sekundy stojí. **Bez korekce je 99. percentil šest milisekund.**

▶ KLIK **S korekcí 1 270**, zhruba dvěstěkrát víc.

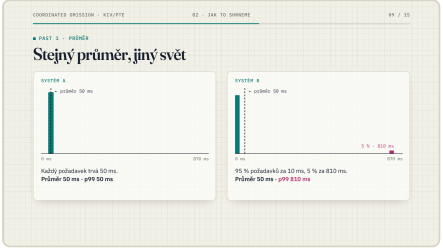
▶ KLIK Rozdíl je jen v tom, že korekce **počítá čas od chvíle, kdy požadavek měl odejít**.



08 5:05–5:10 · ~5 s

Předěl: Jak to shrneme

I při správném měření čekají tři pasti.

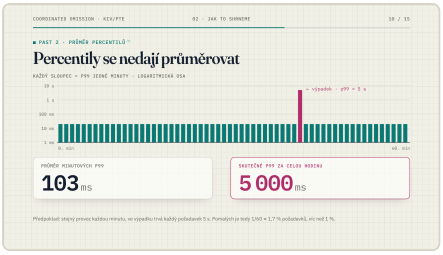


09 5:10–5:50 · ~40 s

Past 1: stejný průměr, jiný svět

První je samotný průměr. Systém A odpoví vždy za padesát milisekund. Systém B v 95 procentech za deset a v pěti procentech za 810. **Průměr mají oba padesát.**

▶ KLIK Ale **99. percentil systému B je 810 milisekund**, protože nejhorší procento leží v těch pomalých pěti. **Proto se hlásí percentily a maximum.**

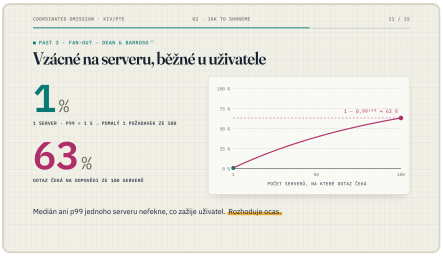


10 5:50–6:50 · ~60 s

Past 2: percentily se nedají průměrovat

Druhá past je skoro v každém dashboardu: 99. percentil za každou minutu a pak jejich průměr za hodinu. Tady má 59 minut dvacet milisekund a jedna minuta pět sekund. Pozor, osa je logaritmická. Průměr vyjde 103 milisekund.

▶ KLIK Jenže ta minuta je šedesátina provozu, asi 1,7 procenta, tedy víc než jedno procento. **Skutečný 99. percentil je pět sekund.** **Percentily průměrovat nejde.** Sčítat se mají histogramy. Histogram je jen počet měření v jednotlivých intervalech, takže se dá sčítat, a percentil se spočítá až ze součtu.



11 6:50–7:45 · ~55 s

Past 3: fan-out (The Tail at Scale)

Třetí past popsali Dean a Barroso z Googlu. Server je pomalý, tedy sekundu a víc, jen v jednom případě ze sta.

▶ KLIK Velké systémy ale rozdělí jeden dotaz na desítky až stovky paralelních volání a odpověď je hotová, až když dorazí to poslední. Při sto serverech je šance, že budou všechny rychlé, je 0,99 na stou, asi 37 procent. Takže **aspoň jeden pomalý je v 63 procentech**.

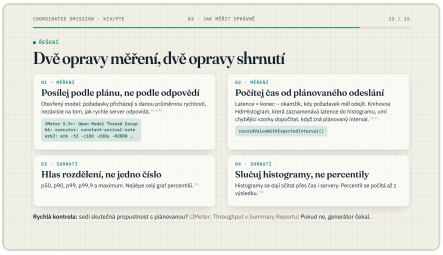
▶ KLIK Pro uživatele tak **rozhoduje ocas.**



12 7:45–7:50 · ~5 s

Předěl: Jak měřit správně

Co s tím?



13 7:50–8:40 · ~50 s

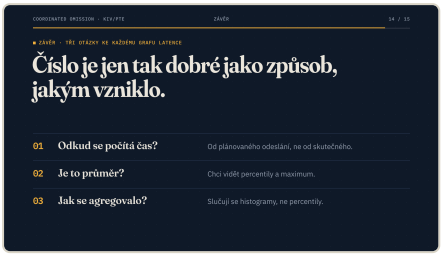
Řešení: dvě opravy měření, dvě opravy shrnutí

První: **posílat zátěž podle plánu, ne podle odpovědí**. V JMeteru to umí **Open Model Thread Group**, která místo počtu vláken plánuje příchody požadavků. Podobně nástroj k6 nebo wrk2.

▶ KLIK Druhá: **počítat čas od plánovaného odeslání**. Knihovna HdrHistogram umí chybějící vzorky dopočítat.

▶ KLIK Třetí: **hlásit percentily a maximum**.

▶ KLIK Čtvrtá: **slučovat histogramy, ne percentily**. A rychlá kontrola: **sedí skutečná propustnost s plánovanou**? Propustnost je počet požadavků za sekundu. Když je nižší, nástroj čekal.



14 8:40–9:55 · ~75 s

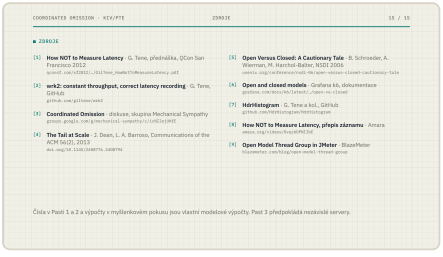
Závěr

Číslo je jen tak dobré jako způsob, jakým vzniklo. Takže u každého grafu latence:

▶ KLIK Odkud se počítá čas? **Má se počítat od chvíle, kdy požadavek podle plánu měl odejít, ne od chvíle, kdy ho nástroj skutečně poslal.** Jinak se čekání ve frontě do výsledku vůbec nedostane.

▶ KLIK Je to průměr? **Pak chtějte vidět percentily a maximum.** Průměr schová ocas rozdělení, tedy právě ty pomalé odpovědi, které uživatelé vnímají nejvíc.

▶ KLIK A jak se data sčítala? **Pokud někdo průměroval percentily po minutách nebo po serverech, výsledné číslo neplatí.** Správně se sčítají celé histogramy a percentil se spočítá až z výsledku.



15 9:55–10:00 · ~5 s

Zdroje a poděkování

Děkuji za pozornost a rád odpovím na otázky.

PŘÍLOHA

Co si nastudovat

Tahle část se neříká nahlas. Je to podklad, abys rozuměl tomu, co stojí za slidy, a zvládl otázky po referátu.

1 · Pojmy v jedné větě

Doba odezvy `response time`	Čas od chvíle, kdy klient požadavek odeslal (správně: *měl* odeslat), do přijetí odpovědi. Zahrnuje **čekání ve frontě i obsluhu**.
Doba obsluhy `service time`	Čas, kdy server požadavek skutečně zpracovává. Tohle měří generátor zasažený coordinated omission.
Propustnost `throughput`	Počet dokončených požadavků za jednotku času (req/s).
Percentil pXX	Hodnota, pod kterou leží XX % měření. p50 = medián; p99 = jen 1 % požadavků je pomalejší.
Ocas rozdělení `tail`	Nejpomalejší odpovědi, typicky nad p99. Průměr a medián o něm nic neřeknou.
Uzavřený model `closed`	Pevný počet klientů (vláken); každý pošle další požadavek až po odpovědi, případně po pauze (think time). **Tempo příchodů závisí na serveru.**
Otevřený model `open`	Požadavky přicházejí s průměrnou rychlostí λ (v článku Schroeder a kol. náhodně, Poissonovým procesem) **nezávisle na tom, jak rychle server odpovídá**. Podle stejného článku není úplně realistický ani jeden model: web nejlépe popisuje **částečně otevřený** model (partly-open), kde uživatelé přicházejí nezávisle, ale během návštěvy čekají na odpovědi.
JMeter · Thread Group	Apache JMeter je open-source nástroj na zátěžové testy. **Thread Group** je základní prvek testovacího plánu: určuje počet vláken (virtuálních uživatelů), náběh (ramp-up) a počet opakování nebo dobu běhu. Každé vlákno prochází plán postupně a **další požadavek pošle až po odpovědi na předchozí**, je to tedy uzavřený model.
wrk2 · k6	Nástroje na zátěžové testy. **wrk2** je Teneho úprava nástroje wrk pro HTTP: drží konstantní propustnost (`-R`) a měří od plánovaného času odeslání. **k6** (Grafana) píše testy v JavaScriptu a otevřený model nabízí přes executory `constant-arrival-rate` a `ramping-arrival-rate`.
Fan-out	Jeden dotaz uživatele se rozdělí na mnoho paralelních volání dalších serverů a odpověď je hotová, až když dorazí ta nejpomalejší. Proto se **vzácný ocas jednoho serveru stane běžným zážitkem uživatele**.
Coordinated omission	Systematická chyba měření: když systém zpomalí, generátor přestane posílat a měřit požadavky, takže **vynechá právě ty nejhorší vzorky**. Pojem zavedl Gil Tene (Azul Systems).
HdrHistogram	Knihovna, která zaznamenává latence do histogramu s velkým rozsahem hodnot a nastavitelnou přesností. Histogram jsou jen počty měření v jednotlivých intervalech, a proto **se histogramy dají sčítat**: sečtou se počty a percentil se spočítá až z výsledku. Percentily samotné takhle sčítat ani průměrovat nejde.

2 · Proč generátor „zpomalí s ním“

Pro uzavřený systém platí **zákon doby odezvy** (Response Time Law) z operační analýzy (Lazowska a kol., *Quantitative System Performance*, oddíl 3.3). V knize je zapsaný jako R = N/X − Z, tady upravený na propustnost:

$$X = N / (R + Z)$$

kde **X** je propustnost, **N** počet klientů (vláken), **R** doba odezvy a **Z** pauza mezi požadavky. Příklad: 200 vláken, Z = 1 s, R = 5 ms → X ≈ 199 req/s. Když se server zasekne a R vyroste, **X automaticky klesne**: generátor posílá méně požadavků přesně ve chvíli, kdy by měl systém zatěžovat nejvíc. To je mechanismus coordinated omission v jednom vzorci.

Zákon doby odezvy plyne z **Littleova zákona**, v knize N = X · R (často také L = λ · W): průměrný počet požadavků v systému = propustnost × průměrná doba v systému. Platí pro otevřený i uzavřený systém, pokud je v ustáleném stavu (kolik požadavků přijde, tolik jich odejde), a hodí se na rychlou kontrolu, jestli čísla z testu dávají smysl.

3 · Jak funguje korekce

- **wrk2** s parametrem `-R` drží konstantní propustnost. Každý požadavek má plánovaný čas odeslání a **latence se počítá od plánu**, ne od skutečného odeslání. Volbou `--u_latency` vypíše i nekorigovaná čísla pro srovnání.
- **HdrHistogram** `recordValueWithExpectedInterval(hodnota, interval)`: když je hodnota větší než očekávaný interval mezi požadavky, doplní navíc hodnoty *hodnota − interval*, *hodnota − 2·interval*, … dokud neklesnou pod interval. Příklad z myšlenkového pokusu: interval 10 ms (100 req/s), naměřeno 100 s → doplní se 9 999 hodnot od 99,99 s dolů po 10 ms. To jsou přesně požadavky, které generátor během výpadku nevyslal.
- Korekce je **odhad**, který předpokládá rovnoměrné příchody. Lepší je měřit rovnou v otevřeném modelu.

4 · Výpočty ze slidů

Myšlenkový pokus, benchmark	(10 000 × 1 ms + 1 × 100 000 ms) / 10 001 ≈ 11,0 ms. Tene na slidu uvádí „10.9 msec“, přesně je to 10,999 ms.
Myšlenkový pokus, uživatelé	10 000 požadavků po 1 ms + 10 000 požadavků čekajících 100 s, 99,99 s, … 0,01 s (průměr ≈ 50 s) → (10 s + 10 000 × 50,005 s) / 20 000 ≈ 25 s. Pro percentily si všech 20 000 hodnot seřaď od nejmenší: prvních 10 000 je rychlých (1 ms), za nimi jsou čekající požadavky od 0,01 s do 100 s. Medián je 10 000. hodnota, tedy poslední rychlá (1 ms). 75. percentil je 15 000. hodnota, tedy prostředek pomalé poloviny (≈ 50 s). 99,99. percentil je 19 998. hodnota, skoro úplně nahoře (≈ 100 s).
wrk2	1 270 ms / 6,04 ms ≈ 210 → README uvádí „200x“.
Past 1	0,95 × 10 ms + 0,05 × 810 ms = 9,5 + 40,5 = 50 ms. p99 systému B leží v horních 5 %, tedy 810 ms.

Past 2	(59 × 20 ms + 5 000 ms) / 60 = 6 180 / 60 = 103 ms. Pomalá minuta = 1/60 ≈ 1,67 % požadavků > 1 % → skutečné p99 = 5 s. Osa grafu je logaritmická, protože 5 s je 250krát víc než 20 ms: na lineární ose by běžné minuty měřily zhruba 1 pixel a nebyly by vidět.
Past 3	Každý server je rychlý s pravděpodobností 0,99. Že je rychlých všech n serverů, je $0{,}99^n$, pro n = 100 asi 37 %. Opak, tedy aspoň jeden pomalý, je $1 - 0{,}99^n$. n = 10 → 9,6 %, n = 100 → 63,4 %. Předpoklad: servery jsou pomalé nezávisle na sobě.

5 · Možné otázky po referátu

Q · Je uzavřený model vždycky špatně?

> Ne. Pokud má reálný systém pevný počet klientů, kteří na odpověď opravdu čekají (dávkové zpracování, pevný počet terminálů), uzavřený model ho popisuje věrně. Problém je, když reálné příchody na rychlosti serveru nezávisí. Schroeder, Wierman a Harchol-Balter ukazují, že **volba modelu výrazně mění výsledky** (průměrná doba odezvy se při stejné zátěži může lišit o řád i víc) a že většina generátorů zátěže a benchmarků přitom používá uzavřený model. Web podle nich leží mezi oběma modely, jako částečně otevřený systém. Volbu modelu tedy musí určovat realita, ne pohodlí nástroje.

Q · Nestačí v JMeteru přidat Constant Throughput Timer?

> Ne. Timer jen brzdí vlákna na cílovou propustnost. Když server stojí, vlákna dál čekají na odpovědi a nové požadavky neodcházejí. Víc vláken problém zmírní, ale neodstraní. Řešením je **Open Model Thread Group** (JMeter 5.5+) a kontrola skutečné propustnosti proti plánu.

Q · Proč nehlásit prostě maximum?

> Maximum je důležité a coordinated omission ho obvykle neschová, protože ten jeden pomalý vzorek se zapíše. Neřekne ale, *jak často* se to děje: jeden výpadek za den, nebo polovina požadavků? Proto **percentily a maximum dohromady**.

Q · Jak dostanu p99 za hodinu, když mám uložené jen minutové p99?

> Přesně nijak, z percentilů se zpět rozdělení nesložíš. Je potřeba ukládat **histogramy** (HdrHistogram, případně přibližné struktury jako t-digest), sloučit je a percentil spočítat až z výsledku.

Q · Proč se v myšlenkovém pokusu nezměnil medián?

> Polovina všech požadavků (prvních 100 s) je rychlá, takže medián leží přesně na hranici a zůstane na 1 ms. Medián o horší polovině neříká vůbec nic.

Q · Jak poznám coordinated omission ve výsledcích?

> Nejrychleji: **skutečná propustnost je nižší než plánovaná** (v JMeteru Throughput v Summary Reportu). Dále „díry“ v časové ose vzorků v době, kdy systém zpomalil.

Q · Dá se s ocasem u fan-outu něco dělat?

> Dean a Barroso navrhují mimo jiné **hedged requests**: když odpověď nepřijde do očekávaného p95, pošle se stejný požadavek ještě jiné replice a použije se ta rychlejší odpověď. Zátěž to zvýší jen asi o 5 % a ocas výrazně zkrátí.

Q · Týká se to jen nástrojů na zátěžové testy?

> Ne. Vznikne všude, kde měřicí smyčka čeká na měřený systém: v mikrobenchmarcích, v klientovi, který si měří vlastní volání, nebo v monitoringu, který vidí jen dokončené požadavky.

---

6 · Doporučené studium

1. **Gil Tene: How NOT to Measure Latency** (přednáška, slidy z QCon SF 2012, přepis na Amara). Základ celého referátu: coordinated omission, percentily, HdrHistogram.
   Slidy: qconsf.com/sf2012/…/GilTene_HowNotToMeasureLatency.pdf
   Přepis: amara.org/videos/0vqzbDPNIIbE
2. **README nástroje wrk2**, část o coordinated omission a příklad s pozastavením serveru přes Ctrl+Z.
   github.com/giltene/wrk2
3. **J. Dean, L. A. Barroso: The Tail at Scale** (Communications of the ACM, 2013). Krátký článek o ocasu latence ve velkých systémech a technikách proti němu.
   doi.org/10.1145/2408776.2408794
4. **B. Schroeder, A. Wierman, M. Harchol-Balter: Open Versus Closed: A Cautionary Tale** (NSDI 2006). Kdy použít otevřený a kdy uzavřený model.
   usenix.org/conference/nsdi-06/open-versus-closed-cautionary-tale
5. **Dokumentace k6: Open and closed models**, stručné vysvětlení s příklady executorů.
   grafana.com/docs/k6/latest/using-k6/scenarios/concepts/open-vs-closed
6. **BlazeMeter: Open Model Thread Group**, jak nastavit otevřený model v JMeteru.
   blazemeter.com/blog/open-model-thread-group
7. **README knihovny HdrHistogram**, záznam s očekávaným intervalem a slučování histogramů.
   github.com/HdrHistogram/HdrHistogram
8. **E. D. Lazowska, J. Zahorjan, G. S. Graham, K. C. Sevcik: Quantitative System Performance** (1984), kap. 3 Fundamental Laws, oddíl 3.3 (Littleův zákon a zákon doby odezvy). Autoři knihu zdarma zveřejnili online. Pro hlubší pochopení, proč se uzavřený systém sám přibrzdí.
   homes.cs.washington.edu/~lazowska/qsp

TAHÁK NA KARTIČKU

Měření: posílat podle plánu · čas od plánovaného odeslání · kontrolovat propustnost.

Shrnutí: percentily + maximum · nikdy neprůměrovat percentily · slučovat histogramy.

Tři otázky ke grafu: Odkud se počítá čas? Je to průměr? Jak se agregovalo?